# Web Data Mining

## Final Project Report

*Yann Lin · François-Louis Legland*

---

**Domain:** Science-Fiction Authors & Works

`v1.0-final`

| | |
|---|---|
| **Stack** | Python 3.12, rdflib, spaCy, PyKEEN, OWLReady2, Ollama (Gemma 2B) |
| **KGE models** | TransE, ComplEx |
| **Reasoner** | Pellet (via OWLReady2) |
| **LLM backend** | Gemma:2b via Ollama (local) |

# Contents


**1 Data Acquisition & Information Extraction** **2**
- 1.1 Domain & Seed URLs . . . 2
- 1.2 Crawler Design & Pipeline . . . 2
- 1.3 NER & Triple Extraction . . . 3
- 1.4 Ambiguity Analysis — Three Entity Ambiguity Cases . . . 3
- 1.5 Scaling Reflection — 10 000 Pages . . . 3

**2 Knowledge Base Construction, Alignment & Expansion** **4**
- 2.1 Initial Graph — Open Library API . . . 4
- 2.2 Alignment Strategy . . . 4
  - 2.2.1 Entity Alignment . . . 4
  - 2.2.2 Predicate Alignment . . . 5
- 2.3 Expansion Strategy . . . 5
- 2.4 Final KB Statistics . . . 6
- 2.5 Discussion . . . 6
  - 2.5.1 Impact of Predicate Alignment Quality . . . 6
  - 2.5.2 Impact of Noisy Expansion . . . 6
  - 2.5.3 Effect of Ontology Modeling Choices . . . 6

**3 Reasoning — SWRL & OWLReady2** **7**
- 3.1 SWRL Rule on `family.owl` . . . 7
- 3.2 SWRL Rule on the Sci-Fi KG . . . 7
- 3.3 Unreasonable Inferences: Symmetric + Transitive Only . . . 7

**4 Knowledge Graph Embeddings** **8**
- 4.1 Data Preparation & Splits . . . 8
- 4.2 Models & Training Configuration . . . 8
- 4.3 Results & Analysis . . . 8
- 4.4 Nearest-Neighbour Analysis . . . 9
- 4.5 SWRL vs. KGE — Rule Capture Analysis . . . 9
- 4.6 How Lab Design Influenced Embedding Performance . . . 10

**5 RAG over RDF/SPARQL** **10**
- 5.1 Architecture Overview . . . 10
- 5.2 Schema Summary & Prompt Template . . . 10
- 5.3 Evaluation — 5 Questions: Baseline vs. RAG . . . 11

**6 Critical Reflection** **12**
- 6.1 KB Quality Impact . . . 12
- 6.2 Noise Issues . . . 12
- 6.3 Rule-Based vs. Embedding-Based Reasoning . . . 12
- 6.4 What We Would Improve . . . 13

# 1. Data Acquisition & Information Extraction

## 1.1 Domain & Seed URLs

The domain chosen for this project is **Science-Fiction literature**, focusing on canonical authors and their works. Eight seed URLs were manually selected from Wikipedia to provide authoritative, richly interlinked content:

**Table 1:** Seed URLs and raw word counts after trafilatura extraction

| Entity | URL | Words |
|---|---|---|
| Isaac Asimov | `en.wikipedia.org/wiki/Isaac_Asimo` | 22 374 |
| Frank Herbert | `en.wikipedia.org/wiki/Frank_Herbert` | 6 454 |
| Arthur C. Clarke | `en.wikipedia.org/wiki/Arthur_C._C` | 13 287 |
| Philip K. Dick | `en.wikipedia.org/wiki/Philip_K._Dick` | 13 358 |
| Ursula K. Le Guin | `en.wikipedia.org/wiki/Ursula_K._L` | 14 608 |
| Science fiction | `en.wikipedia.org/wiki/Science_fiction` | 13 925 |
| Dune (novel) | `en.wikipedia.org/wiki/Dune_(novel` | 16 577 |
| Victor Hugo (FR) | `fr.wikipedia.org/wiki/Victor_Hugo` | 26 808 |
| **Total** | | **127 791** |

All 8 pages exceeded the 500-word quality threshold and were retained in `crawler_output.jsonl`.

## 1.2 Crawler Design & Pipeline

The crawling pipeline uses **httpx** for HTTP fetching and **trafilatura** for content extraction. Trafilatura was chosen because it performs boilerplate removal, main-content extraction, and encoding normalisation in a single call, producing clean UTF-8 paragraphs directly suitable for NER.

The pipeline is structured as follows:

- **Fetch:** `httpx.get()` with a custom `User-Agent` header and a 10 s timeout. HTTP errors (4xx/5xx) are caught and logged; the URL is skipped.
- **Extract:** `trafilatura.extract()` strips navigation, footers, scripts, and ads, returning only main article text.
- **Quality gate:** pages with fewer than 500 words are discarded to avoid stub content.
- **Persistence:** each page is appended as a JSON line to `crawler_output.jsonl`, keyed by URL and text.

**Ethics:** Wikipedia's `robots.txt` permits automated crawling for non-commercial research. Requests are sequential (no parallelism) and carry an explicit `User-Agent` string identifying the academic project.

## 1.3 NER & Triple Extraction

The NER model used is spaCy's `en_core_web_trf`, a transformer-based pipeline built on the RoBERTa architecture, selected for its higher precision on entity boundaries compared to statistical models. Text was truncated to 5 000 characters per page to stay within the transformer's context window.

Entity types retained: `PERSON`, `ORG`, `GPE`, `WORK_OF_ART`, `DATE`. This yielded **558 unique entities** from the 8 pages.

Triple extraction used dependency parsing: for each sentence containing at least two named entities, verbs with an `nsubj`/`nsubjpass` subject and a `dobj`/`pobj` object were identified as candidate predicates. This produced **135 raw subject–verb–object triples**:

```
subject        relation        object
Asimov         write           mysteries
Asimov         write           series
Hugo           publie          roman
Foucher        rapproche       couple
Adele          avoue           infidelite
```

## 1.4 Ambiguity Analysis — Three Entity Ambiguity Cases

### Case 1: "Petrovichi" labelled as PERSON instead of GPE

In the Isaac Asimov Wikipedia page, the village of *Petrovichi* (Asimov's birthplace in Russia) was labelled `PERSON` by `en_core_web_trf`. The model was likely misled by the Slavic suffix *-ichi*, which superficially resembles a genitive personal name form. The correct label is `GPE` (location). This produced a false entity in `extracted_knowledge_scifi.csv`, visible in the first rows of the entity table.

### Case 2: "Arthur C. Clarke.[2" — truncated citation as entity

The string `Arthur C. Clarke.[2` was extracted as a `PERSON` entity. This is a citation artefact: Wikipedia footnote markers (`[2]`) were not fully stripped by trafilatura when they immediately follow a name. The result was a distinct entity that could not be aligned to DBpedia (appearing as "Not Found" in the entity alignment table), polluting the entity list with a near-duplicate. The fix is a post-extraction regex to strip trailing `[\d+]` footnote markers.

### Case 3: "72" labelled as DATE in Asimov's article

The number `72` (Asimov's age at death) was tagged as a `DATE` entity. While technically a temporal quantity, it is not an absolute date — it is a duration/age. This produced a spurious triple `(Isaac_Asimov, age, 72)`, which was retained in the entity CSV. The distinction between `CARDINAL` and `DATE` is a known weakness of transformer NER models when applied to bare integers without surrounding temporal context words.

## 1.5 Scaling Reflection — 10 000 Pages

At 8 pages the full pipeline runs sequentially in under 2 minutes. Scaling to 10 000 pages requires rethinking every component:

- **Crawling:** Replace the sequential `httpx` loop with a Scrapy-Cluster or Apache Nutch setup using multiple worker processes. Implement a Redis-backed URL frontier with per-domain politeness queues.
- **Content extraction:** trafilatura is CPU-bound but highly parallelisable; deploy as a containerised microservice behind a Kafka topic.
- **NER at scale:** `en_core_web_trf` processes ≈ 1 page/s on CPU — 10 000 pages ≈ 3 hours. Solutions: switch to `en_core_web_lg` (5× faster), use GPU inference via `spacy[cuda]`, or adopt batched streaming inference (`nlp.pipe` with `batch_size=64`).
- **Deduplication:** Replace in-memory seen-URL sets with a Bloom filter (`pybloom-live`) or Redis SET.
- **Storage:** Stream triples directly into N-Triples or a graph database (Blazegraph, GraphDB) rather than accumulating in a Python list.
- **Monitoring:** Add structured logging and Prometheus metrics for crawl rate, NER throughput, and triple extraction rate.

# 2. Knowledge Base Construction, Alignment & Expansion

## 2.1 Initial Graph — Open Library API

The initial knowledge graph (`my_private_kg`) was constructed by querying the Open Library Subjects API for the top 35 science-fiction works. For each work, the following triple patterns were generated:

- `(book_uri, rdf:type, priv:Book)`
- `(book_uri, pred:title, Literal(title))`
- `(book_uri, pred:firstPublishYear, Literal(year))`
- `(author_uri, rdf:type, priv:Author)`
- `(author_uri, pred:wroteBook, book_uri)`

This yielded an initial graph of **188 triples covering 61 entities**.

## 2.2 Alignment Strategy

### 2.2.1 Entity Alignment

Entity alignment was performed in two stages.

**Stage 1 — Seed entities:** The 7 seed Wikipedia URLs were directly mapped to DBpedia URIs by replacing `en.wikipedia.org/wiki/` with `http://dbpedia.org/resource/`.

Each candidate URI was verified with a SPARQL `ASK` query against the DBpedia endpoint. All 7 seed entities were confirmed at confidence 1.0.

**Stage 2 — NER entities:** The 236 entities extracted in TD1 (`PERSON` and `WORK_OF_ART` types) were aligned using the same DBpedia ASK approach. **168 out of 236 entities were matched (71.2%)**. The full mapping was saved to `entity_alignment.csv`.

**Table 2:** Sample of entity alignment results

| Private Entity | External URI | Confidence |
|---|---|---|
| Isaac_Asimov | `dbpedia.org/resource/Isaac_Asim` | 1.0 |
| Frank_Herbert | `dbpedia.org/resource/Frank_Herbert` | 1.0 |
| Arthur_C._Clarke | `dbpedia.org/resource/Arthur_C._` | 1.0 |
| Robert A. Heinlein | `dbpedia.org/resource/Robert_A._Heinlein` | 1.0 |
| Petrovichi | `dbpedia.org/resource/Petrovichi` | 1.0 |
| Chas. Reg. Dawson | Not Found | 0.0 |
| Arthur C. Clarke.[2 | Not Found | 0.0 |

### 2.2.2 Predicate Alignment

Predicate alignment was performed by querying DBpedia's ontology for keywords matching the private predicates, then applying the best semantic match via OWL axioms:

**Table 3:** Predicate alignment decisions

| Private Predicate | DBpedia Alignment | Relation | OWL Axiom |
|---|---|---|---|
| `pred:title` | `dbo:title` | Exact match | `owl:equivalentProperty` |
| `pred:genre` | `dbo:genre` | Exact match | `owl:equivalentProperty` |
| `pred:firstPublish` | `dbo:activeYears` | Sub-property | `rdfs:subPropertyOf` |
| `pred:name` | `dbo:alternativeName` | Sub-property | `rdfs:subPropertyOf` |
| `pred:wroteBook` | `dbo:author` | Inverse | `owl:inverseOf` |

## 2.3 Expansion Strategy

The knowledge base was expanded via a **2-hop SPARQL CONSTRUCT** query against DBpedia. The anchor set consisted of 300 Sci-Fi writers retrieved with:

```
SELECT DISTINCT ?author WHERE {
  ?author a dbo:Writer .
  ?author dbo:genre dbr:Science_fiction .
} LIMIT 300
```

For each anchor, one-hop triples (author properties) and two-hop triples (author → `dbo:notableWork` → work properties) were fetched. Noisy predicates were explicitly excluded: `dbo:abstract`, `wikiPageUsesTemplate`, `prov:wasDerivedFrom`, `foaf:isPrimaryTopicOf`. Each request was capped at 6 000 triples with a 0.1 s inter-request delay.

The expansion reached **75 603 triples (394 predicates)** before cleaning. A predicate frequency filter retained the top 150 most frequent predicates and restricted the graph to URI→URI triples only, reducing the graph to **57 828 triples and 80 predicates**.

## 2.4 Final KB Statistics

**Table 4:** Final knowledge base statistics

| Metric | Value |
|---|---|
| Total triples (final clean graph) | 57 996 |
| Triples after DBpedia-only filter (KGE) | 48 603 |
| Number of entities (full) | 31 481 |
| Number of entities (DBpedia-only) | 26 439 |
| Distinct predicates (after cleaning) | 80 (from 394) |
| Ontology classes (private) | 3 (`priv:Book`, `priv:Author`, `priv:SciFiFanzine`) |
| `owl:sameAs` alignment links | 168 |
| Train / Valid / Test (full) | 51 671 / 3 178 / 3 147 |
| Train / Valid / Test (DBpedia-only) | 43 149 / 2 719 / 2 735 |

## 2.5 Discussion

### 2.5.1 Impact of Predicate Alignment Quality

The predicate alignment was correct but applied only to 5 private predicates. The 80 predicates in the final graph are dominated by raw DBpedia predicates from the expansion. The coexistence of `dbo:genre` and `dbp:genre` (same relationship, different namespaces) means the model treats them as two separate relation types with two separate embedding vectors, diluting the training signal for each.

### 2.5.2 Impact of Noisy Expansion

The 2-hop DBpedia expansion introduced parasitic entities: web archive URLs (`web.archive.org`), national library identifiers (`d-nb.info`), and opaque identifiers such as `2ujL4`. These appeared in the nearest-neighbour output for Isaac Asimov. The strict `dbpedia.org` domain filter reduced the entity count from 31 481 to 26 439 but did not eliminate non-English DBpedia mirror URIs (`af.dbpedia.org`, etc.).

### 2.5.3 Effect of Ontology Modeling Choices

Using two separate namespaces (`PRIV` for entities, `PRED` for predicates) created a dual-identity problem: `priv:Isaac_Asimov` and `dbr:Isaac_Asimov` both existed in the graph, connected only by `owl:sameAs`. Since `TriplesFactory` in PyKEEN does not resolve `owl:sameAs` by default, the model learned independent vectors for both, effectively halving the training signal for each entity representation.

# 3. Reasoning — SWRL & OWLReady2

## 3.1 SWRL Rule on `family.owl`

The SWRL rule applied to `family.owl` classifies any `Person` older than 60 as an `oldPerson`:

```
Person(?p), age(?p, ?age), greaterThan(?age, 60) ->
   oldPerson(?p)
```

Before reasoning, no individuals were assigned to `oldPerson`. After running Pellet via OWLReady2, two individuals were correctly classified:

- **Peter** → classified as `oldPerson` (age: 70)
- **Marie** → classified as `oldPerson` (age: 69)

Pellet also reclassified `Marie` by removing redundant `Female` membership (subsumed by `Mother`), and reclassified `Grandfather` by removing redundant `Male` membership.

## 3.2 SWRL Rule on the Sci-Fi KG

A domain-specific SWRL rule was defined to infer `SciFiAuthor` status:

```
Writer(?w), notableWork(?w, ?book) -> SciFiAuthor(?w)
```

This rule captures the intuition that any DBpedia `Writer` with at least one notable work catalogued in the graph is a Science Fiction author in our domain context. In practice, this fired for all 300 anchor entities (who are already of type `dbo:Writer` and linked to works via `dbo:notableWork`).

## 3.3 Unreasonable Inferences: Symmetric + Transitive Only

If only **Symmetric** and **Transitive** axioms are applied to a property such as `isSiblingOf`, three classes of unreasonable inferences arise:

**Problem 1 — Spurious Reflexivity**

$$A \text{ isSiblingOf } B \xrightarrow{\text{Symm.}} B \text{ isSiblingOf } A \xrightarrow{\text{Trans.}} A \text{ isSiblingOf } A$$

Every individual becomes their own sibling.
To fix this, we declare `isSiblingOf` as `owl:IrreflexiveProperty`.

**Problem 2 — Cross-Family Contamination**

If a noisy triple asserts `Paul isSiblingOf John` (where John is not a sibling of Paul), transitivity immediately yields `Peter isSiblingOf John` for any Peter linked to Paul. In a large graph, transitivity chains across the entire population, merging all individuals into a single sibling equivalence class.

**Problem 3 — Generational Collapse**

From `family.owl`: Thomas `isChildOf` Peter, and Peter `isSiblingOf` Paul (confirmed by Q12). If `isSiblingOf` is also made transitive and additional cross-generation paths exist, Thomas can become a sibling of his uncle's children, collapsing the generational hierarchy.
The correct modelling would be to express siblinghood via the role chain `isChildOf` $\circ$ `isParentOf` $\sqsubseteq$ `isSiblingOf`, which naturally limits transitivity to the same parental generation.

# 4. Knowledge Graph Embeddings

## 4.1 Data Preparation & Splits

Before training, the full graph (57 996 triples) was filtered to retain only URI→URI triples where both subject and object URIs belong to the `dbpedia.org` domain, yielding **48 603 clean triples** over **26 439 entities and 71 relations**.

Splits used a 80/10/10 ratio with an entity-safety correction: any validation or test triple containing an entity unseen in the training set was moved to the training set. After correction, zero entity leakage was confirmed.

## 4.2 Models & Training Configuration

Two models were trained using PyKEEN on CPU:

- **TransE:** translational model, learns entity and relation vectors such that $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$.
- **ComplEx:** complex-valued model, effective for symmetric and antisymmetric relations.

Common configuration: `embedding_dim=50`, `num_epochs=30`, `batch_size=256` (reduced to 32 by PyKEEN's automatic CPU memory guard), `lr=0.01`, `random_seed=42`.

## 4.3 Results & Analysis

**Table 5:** KGE evaluation results on the test set

| Metric | TransE | ComplEx | Notes |
|---|---|---|---|
| MRR | 0.0004 | 0.0004 | Very low — noise-dominated |
| Hits@1 | 0.0000 | 0.0000 | No top-1 correct predictions |
| Hits@3 | 0.0002 | 0.0000 | Marginal for TransE |
| Hits@10 | 0.0004 | 0.0004 | Both models at chance level |

Both models converged to near-zero metrics. The root causes are:

1. Very large entity space (26k entities) relative to training triples (43k), yielding an extremely sparse graph.
2. Only 30 epochs with `batch_size=32` (CPU memory guard), meaning each entity was seen very few times per epoch.
3. Residual noisy triples (non-English DBpedia mirrors, archive URLs) despite the DBpedia-only filter.

## 4.4 Nearest-Neighbour Analysis

Despite low ranking metrics, the ComplEx embeddings were probed for nearest neighbours of Isaac Asimov:

```
Target: http://af.dbpedia.org/resource/Isaac_Asimov

Top 10 nearest neighbours (cosine similarity):
  0.4129 | Category:Writers_from_Odesa
  0.3935 | Twilight_(Meyer_novel)
  0.3853 | Neanderthal_Planet_(short_story)
  0.3796 | Category:Scientists_from_New_York_City
  0.3742 | Maurice_Leblanc
  0.3727 | Digital_rights_management
  0.3650 | Locus_Award_for_Best_Novelette
  0.3607 | Defense_Command_(book_series)
  0.3585 | Resonator
  0.3515 | 2ujL4
```

Some neighbours are plausible (*Category:Writers_from_Odesa* reflects Asimov's birthplace; *Neanderthal_Planet* is a real Asimov story; *Locus_Award_for_Best_Novelette* is domain-relevant). However, the presence of *Digital_rights_management*, *Resonator*, and the opaque identifier `2ujL4` confirms that parasitic expansion nodes contaminated the embedding space.

## 4.5 SWRL vs. KGE — Rule Capture Analysis

The cosine similarity between the `notableWork` and `genre` relation embeddings was measured to check whether the SWRL rule

$$\texttt{Writer}(w) \wedge \texttt{notableWork}(w, b) \;\Rightarrow\; \texttt{SciFiAuthor}(w)$$

is captured in the embedding space:

```
Cosine similarity notableWork <-> genre : -0.1198
-> The two relations are DISTANT in the vector space.
   The embedding does NOT capture the SWRL rule.
```

This confirms the fundamental tension between rule-based and embedding-based reasoning: SWRL requires exact logical co-occurrence of two properties, while KGE learns a

continuous geometry that with only 30 noisy training epochs has not converged to reflect the semantic co-implication.

### 4.6 How Lab Design Influenced Embedding Performance

- **Predicate explosion (Lab 2):** The 394 raw predicates were reduced to 80 by frequency filtering. Keeping the top-150 by frequency rather than by semantic relevance may have discarded rare but important predicates (`dbo:influenced`, `dbo:mainInterest`) while retaining high-frequency noise (`wikiPageWikiLink` covered ≈50% of triples).
- **Dual namespace (Lab 2):** `PRIV` and DBpedia entities for the same person coexisted as separate nodes. Without `owl:sameAs` fusion before creating `TriplesFactory`, the model learned two independent vectors per real-world entity.
- **Parasitic entities (Lab 2):** Despite `dbpedia.org` filtering, Afrikaans (`af.dbpedia.org`) and other language mirror URIs were included. A stricter filter to `en.dbpedia.org` only would have been more precise.
- **CPU constraint:** The `batch_size=32` fallback is a direct consequence of running on CPU. Hub entities like Isaac Asimov (many `wikiPageWikiLink` triples) are relatively well-trained, but tail entities appear too rarely for the model to learn useful representations.
- **Ontology design (Lab 3):** No typing information from the OWL ontology was injected into the KGE training. Type-constrained negative sampling was therefore not possible.

## 5. RAG over RDF/SPARQL

### 5.1 Architecture Overview

The RAG pipeline (TP6) loads the graph (57 996 triples, serialised as `scifi_kg.ttl`) into an rdflib `Graph` at startup. The LLM backend is Ollama running `gemma:2b` locally. The pipeline has three layers:

1. **Layer 1 — LLM generation:** the schema summary and few-shot prompt are sent to Ollama to generate a SPARQL SELECT query.
2. **Layer 2 — Self-repair:** if the generated query raises an rdflib `ParseException` or evaluation error, the error message is fed back to the LLM with the instruction to fix the query.
3. **Layer 3 — Template fallback:** if both LLM attempts fail, a deterministic template is applied based on keyword matching (`works`, `born in`, `science fiction`, `genre`, `movement`).

### 5.2 Schema Summary & Prompt Template

The NL→SPARQL prompt template is:

```
You are a SPARQL generator. Return ONLY a SPARQL SELECT query
in a ```sparql block. Never write explanations.
Use ONLY dbo: and dbr: prefixes. Never use wdt: or wd:.

SCHEMA SUMMARY:
{schema}

EXAMPLES:
Question: What are the works of Isaac Asimov?
```sparql
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>
SELECT ?work WHERE {
  dbr:Isaac_Asimov dbo:wikiPageWikiLink ?work .
} LIMIT 10
```

Now answer ONLY with a SPARQL query:
Question: {question}
```sparql
```

## 5.3 Evaluation — 5 Questions: Baseline vs. RAG

**Table 6:** RAG evaluation results (Gemma 2B + SPARQL vs. baseline)

| Question | Method | Base OK | RAG OK | Results |
|---|---|---|---|---|
| Works of Isaac Asimov? | LLM | No | Partial | 10 |
| Born in Chicago? | LLM | No | Yes | 11 |
| Authors in Sci-Fi genre? | Template fallback | No | Yes | 20 |
| Genre of Gordon R. Dickson? | Template fallback | **Wrong** | Yes | 2 |
| Movement of Isaac Asimov? | LLM | No | Yes | 1 |

The baseline (Gemma 2B without graph access) failed all 5 questions: it hallucinated Gordon R. Dickson's genre as "post-apocalyptic fiction", refused to answer birthplace and movement questions citing lack of real-time data, and produced only a generic author list for the Sci-Fi genre question. The RAG pipeline returned grounded, graph-sourced answers for all 5 questions.

The "works" question returned `wikiPageWikiLink` targets rather than proper `dbo:notableWork` results — a known limitation: `wikiPageWikiLink` captures all outgoing links, not just literary works.
```

# 6. Critical Reflection

## 6.1 KB Quality Impact

The quality of the KB at each stage propagated forward and set a ceiling on downstream performance. The 558 entities extracted in TD1 included artefacts (`Arthur C. Clarke.[2, 72)` that survived into TP4's entity alignment, contributing to the 68 unresolved entities. These missing alignments meant fewer `owl:sameAs` links, leaving the KGE model with disconnected entity clusters rather than a fully bridged graph.

The Open Library API (initial 188 triples) and the Wikipedia NER extraction (135 triples) were small compared to the DBpedia expansion (57 000+ triples). The ontological design choices (`pred:` namespace, custom classes) had minimal impact on the final embedding because the model overwhelmingly learned from DBpedia structure.

## 6.2 Noise Issues

- **NER noise (TD1):** Footnote artefacts, misclassified locations, and numeric ages tagged as DATE created 68 unresolvable entities.
- **Expansion noise (TP4):** Web archive URLs, national library identifiers, and opaque identifiers appeared as entity nodes. The strict `dbpedia.org` URI filter reduced entities from 31 481 to 26 439 but did not eliminate non-English DBpedia mirror URIs.

## 6.3 Rule-Based vs. Embedding-Based Reasoning

**Table 7:** SWRL vs. KGE comparison (from TD5)

| Aspect | SWRL | KGE |
|---|---|---|
| Expressivity | Exact logical conditions | Vectorial proximity (approx.) |
| Data required | Ontology + instances | Large training set |
| Noise tolerance | None (strict logic) | Tolerant (learned from patterns) |
| New facts | Deduction only | Induction possible |
| Interpretability | High (readable rules) | Low (opaque vectors) |

The measured cosine similarity between `notableWork` and `genre` ($-0.1198$) confirms that the KGE model did not capture the SWRL implication chain. This is expected: SWRL requires exact co-occurrence of properties to fire, while KGE learns a geometry from statistical co-occurrences in the triple set. A hybrid approach — using OWL reasoning to close obvious logical gaps and KGE for uncertain, statistical inferences — is preferable to either alone.

## 6.4 What We Would Improve

- **Training:** use a GPU to remove the `batch_size=32` constraint; increase to 100+ epochs with early stopping on validation MRR.
- **NER pipeline:** add post-extraction cleaning (strip footnote markers, filter numeric-only entities); consider switching to a multilingual model (`fr_core_news_lg`) for the French Victor Hugo page.
- **Predicate deduplication:** before KGE, merge equivalent predicates identified by `owl:equivalentProperty` (`dbo:genre` and `dbp:genre`) into a single canonical URI.
- **RAG:** replace `wikiPageWikiLink` with `dbo:notableWork` in the few-shot examples to return actual book titles rather than all outgoing links.